

pyJvsip Python Module
A Proof of Concept
Demonstrating a more Python like VSIPL

Randall Judd

©2026 Randall Judd 2026, all rights reserved.

A non-exclusive, non-royalty bearing license is hereby granted to all persons to copy, modify, distribute and produce derivative works for any purpose, provided that this copyright notice and following disclaimer appear on all copies:

THIS LICENSE INCLUDES NO WARRANTIES, EXPRESSED OR IMPLIED, WHETHER ORAL OR WRITTEN, WITH RESPECT TO THE SOFTWARE OR OTHER MATERIAL INCLUDING, BUT NOT LIMITED TO, ANY IMPLIED WARRANTIES OF MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE, OR ARISING FROM A COURSE OF PERFORMANCE OR DEALING, OR FROM USAGE OR TRADE, OR OF NON-INFRINGEMENT OF ANY PATENTS OF THIRD PARTIES. THE INFORMATION IN THIS DOCUMENT SHOULD NOT BE CONSTRUED AS A COMMITMENT OF DEVELOPMENT BY ANY PARTY.

Introduction

This document describes a signal processing module for python using the C VSIPL specification as a guide. The python VSIPL module is used for most of the computing however that is not necessarily a requirement. C VSIPL has specifications which the author tries to abide by; however there is no specification for python VSIPL so we go our own way.

Although I do not feel bound by the C VSIPL specification I have tried to create a python module that can use any C VSIPL library implementation as the computation module as long as it contains the same functionality as the library I use.

This is a proof of concept; not a final product. Just something I work on in retirement. Much has changed in the compute world since the VSIPL specification was written so my work may not be of much interest.

Installation

The **pyJvsip** module comes as part of the JVSIP VSIPL distribution. For the knowledgeable there are many ways to create the products. This is the default way. Note I just work from a terminal window with standard unix style commands. For other platforms there are probably other ways to do it.

Python has become a bit more complicated since I originally developed this code. I recommend creating a python virtual environment and developing in there.

After download from github there should be a directory **jvsip**. In a terminal window change to that directory and type **make**. This should make the C VSIPL library.

Next **cd** into the **python** directory. This contains several directories but for this document we are only interested in the **vsip** directory and the **pyJvsip** directory. Install the **vsip** module using available instructions. Change into the **pyJvsip** directory and install the module using the command

```
python -m pip install .
```

Fundamentals

Here we discuss some fundamental principals of **C VSIPL** and how they relate to **pyJvsip**

C VSIPL

In C VSIPL the fundamental objects are **blocks** and **views**. These are just structures defined in the specification as incomplete types. An incomplete type is one where the type is defined but the actual structure definition is not included as part of the library definition. You create the structure using supplied creation functions but the library vendor (creator) is allowed to hide the exact definition.

There are other incomplete typedefs in **C VSIPL** for some functionality such as for FFTs or decomposition objects such as QR decomposition of singular value decomposition. These all must be created and destroyed.

So, for instance, you create a C VSIPL block using the block create function. For example

```
vsip_length alength = 1000;
vsip_memory_hint ahint = VSIP_MEM_NONE;
vsip_block_f *ablock = vsip_blockcreate_f(alength, ahint);
```

creates a block and returns a pointer to the object. If you try

```
vsip_length alength = 1000;
vsip_memory_hint ahint = VSIP_MEM_NONE;
vsip_block_f ablock;
```

you will get `error: variable has incomplete type`.

The VSIP forum wanted a pseudo object oriented interface with freedom for vendors to produce code without explicitly showing their source. So **C VSIP** has a lot of create and destroy functions and what might be a method in a true object oriented language is a function defined to operate on these VSIP created objects.

pyJvsip

In **pyJvsip**, in general although not a rule, all the incomplete typedefs in **C VSIP** become classes. The transfer of a class from **C VSIP** to **python** is not one to one. For instance **pyJvsip** has a **Block** class. An instance of the Block class will contain a C VSIP block but we are free to do many things with a Block class. For instance in C VSIP there is no function to get the size (number of elements) of the block after it is created. In **pyJvsip** we can do

```
from pyJvsip import Block
aBlock = Block('block_f',100)
aBlock.length
```

Note that you need to pass in the type of block that you are creating and the size of block you want. Block objects are immutable after they are created, the same as in **C VSIP**.

We also have **views** in **pyJvsip**. In **C VSIP** the view is created and the **block** it is associated with is an argument to the **bind function**. Since views are always associated with a block and may not be reassigned to another block we decided to put the view creation in as a **method** on a block object. So, for instance

```
from pyJvsip import Block
aBlock = Block('block_f',100)
offset = 0; stride = 1; length = 10
aView = aBlock.bind(offset, stride, length)
```

Note that the argument list defines the type of view you want and the block type defines the precision of the view; in this case a vector view of precision float.

We have a lot of bootstrap methods to create objects but we have also defined a create function as a shortcut. So if you just want a simple vector of length 10 you can do

```
from pyJvsip import create
aView = create('vview_f',10)
```

The create function will create many objects other than **views**. To use it you use a type string and an argument list which determines what is created.

Not that a **view** must be associated with a **block**. View objects created directly create an associated block of the exact size needed and maintain an instance variable of the block; otherwise the block would be garbage collected.

You may create a new **view** of the same type and size on a new data space (block) using an **empty method** such as

```
from pyJvsip import create
aView = create('vview_f',10)
```

```
aNewView = aView.empty
```

You may create a new **view** of the same type and size on a new data space (block) using an **copy method** such as

```
from pyJvsip import create
aView = create('vview_f',10)
aNewView = aView.copy
```

The copy method will copy over the data from the calling view. The empty method will return a view with no data.

You may create a new **view** of the same data using an **clone method** such as

```
from pyJvsip import create
aView = create('vview_f',10)
aNewView = aView.clone
```

The new view is on the same data space (block) as the calling view but it is a new view. You may manipulate its attributes without changing the attributes on the calling view. Note that changing data in the new view will also change data in the calling view if the data spaces overlap.

A caution. In **C VSIPL** all objects are explicitly created and destroyed. You need to be careful to not create memory leaks by reassigning variables without destroying the object it references. In **pyJvsip** the python garbage collector will destroy any object that does not have an explicit reference and underlying a lot of functionality objects are being created and destroyed on the fly. You should not have problems with memory leaks but you might accidentally destroy something before you want.

Technique

In general **pyJvsip** uses python dictionaries extensively. The basic key is usually a string corresponding to a C VSIPL type. The value selected by the key is frequently a python VSIP function call. Most of this happens under the cover of the implementation. The user needs these keys if creating a new object but the calculations will check and select proper types at run time with no additional input needed from the user.

Example

If you have a **block** object and want a new block of the same size you can use the empty method. If you want to do something more complicated and want a new block of the same type but not the same size then you can do

```
aNewBlock = Block{oldBlock.type,NewBlockSize}
```

All **pyJvsip** created objects have a property to return their type.